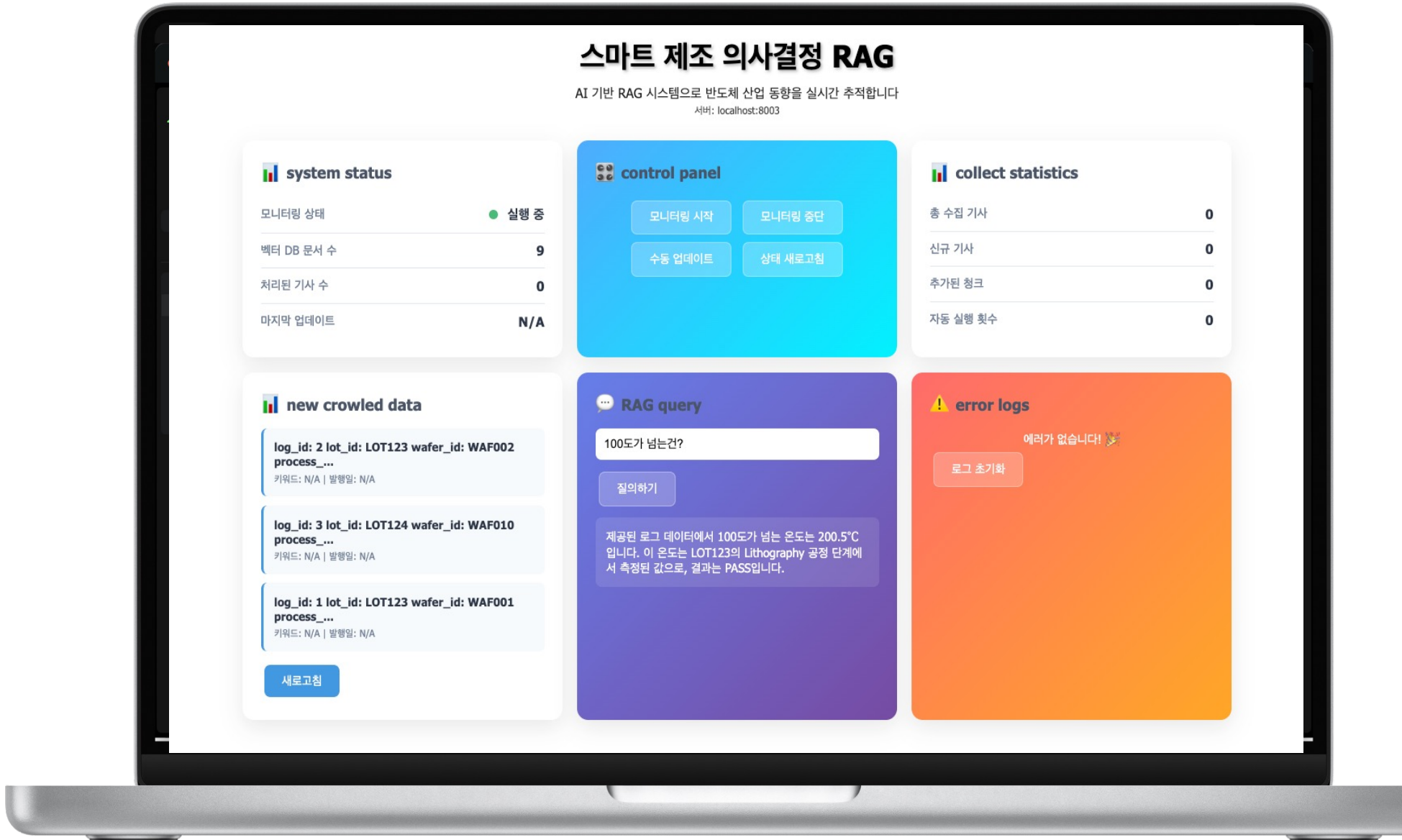


생산공정 이상과 공급망 리스크를 동시에 잡는 스마트 제조 의사결정 RAG, SKAG



시장 사례

"제조업 AI 전환(AX)의 시대"...SK하이닉스, 생성형 AI 적용해 산업 현장 업무 자동화 구현 추진



주) THEELEC, Sk 하이닉스, 제조업 전용 MSP 사업 전면 확대

- 제조 AX 혁신, AI 엔지니어 에이전트
- PLC 코파일럿, RAG 기반 자동화

[도입 배경]

생산공정 복잡화

- 반도체·전자부품 등 정밀 제조업에서는 생산 라인별 이상 징후를 빠르게 식별하기 어려움

공급망 불확실성 확대

- 원자재 수급, 물류 지연, 규제 이슈 등 외부 요인이 실시간으로 생산성과 품질에 영향을 미치고 있음

단일 데이터 소스 한계

- 기존 시스템은 내부 DB만 분석하거나 외부 리스크를 별도로 모니터링하는 구조로, 종합적인 의사결정 지원이 어려움

[시장의 필요성]

비용 압박 심화

- 글로벌 경기 둔화와 원자재 가격 상승으로 제조업 전반의 비용 부담 증가
불량·재작업·리콜로 인한 손실을 줄일 수 있는 기술 도입 필요성 확대

실시간 대응 속도 요구

- 공급망 리스크가 예측 불가능하게 발생 (물류 지연, 지정학적 분쟁 등)
- 사후 보고 체계로는 늦음 → 실시간 데이터 기반 의사결정 체계 필요

정책·규제 환경 변화

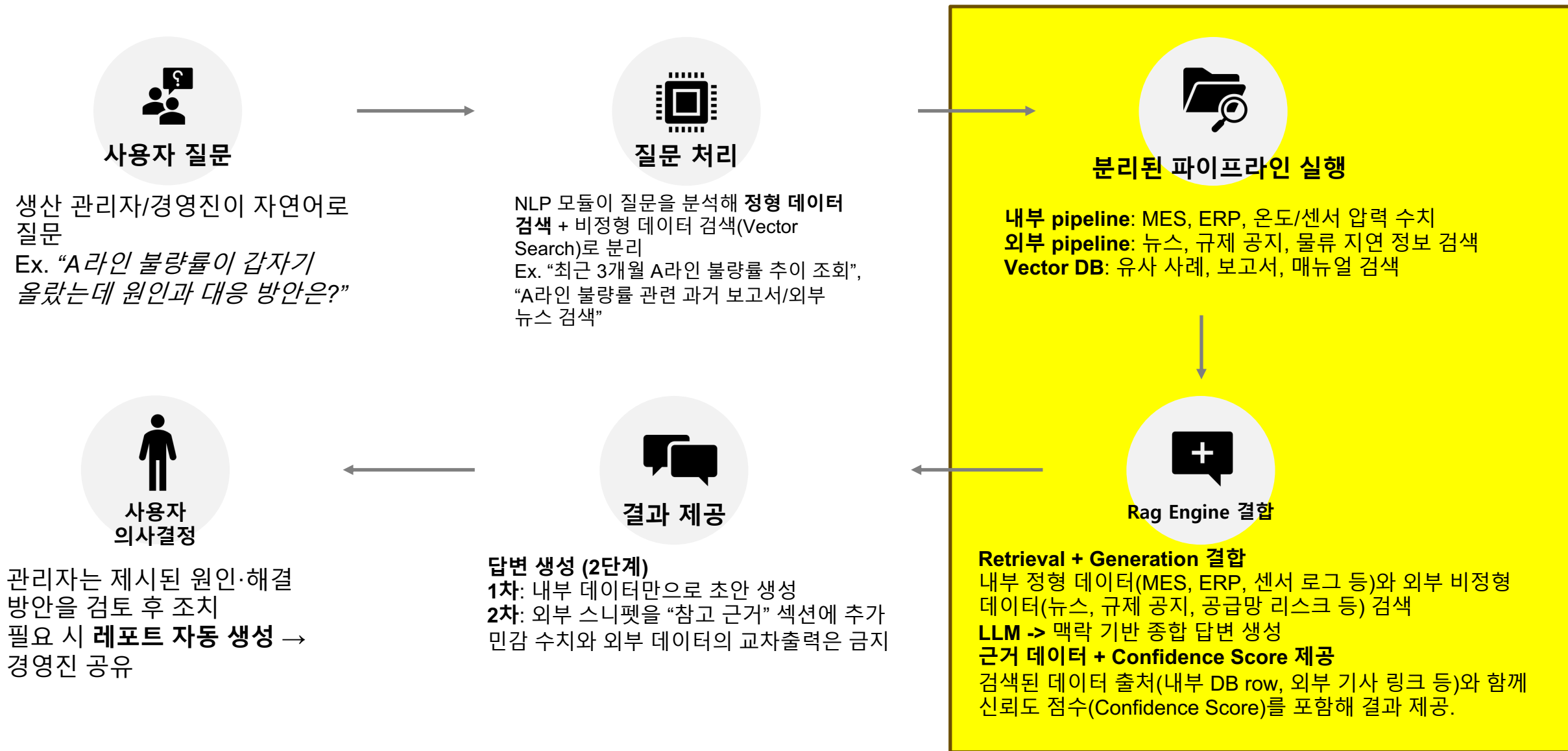
- 정부의 스마트팩토리·AI팩토리 확산 정책과 대규모 투자(수조 원 규모) 추진
ESG, 품질·안전 규제 강화로 데이터 기반 경영 관리 시스템 필요성 증대

경쟁사 동향 가속화

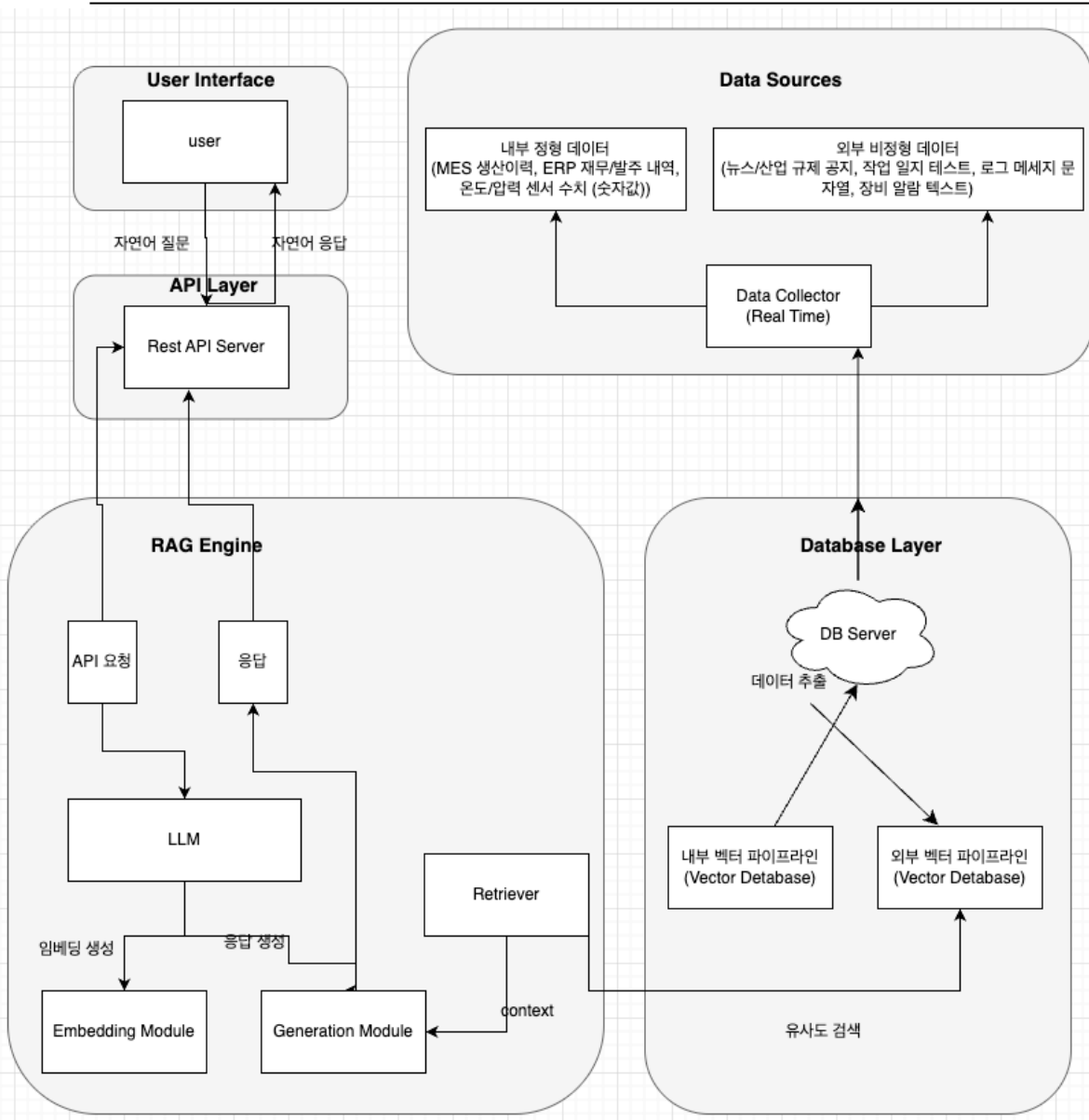
- 포스코, 삼성전자 등 주요 제조사들은 이미 AI·RAG 기반 시스템 시범 적용
- 시장 초기 단계에서 선제적 도입 시 레퍼런스 확보 + 경쟁 우위 선점 가능

service flow

생산공정 이상과 공급망 리스크를 동시에 잡는 **스마트 제조 의사결정 RAG**



스마트 제조 의사결정 RAG 아키텍처



[Data Sources]

- 내부 정형 데이터: MES, 생산 이력, ERP 재무/발주 내역, 온도/압력 센서 수치 (숫자값) -> 1분마다 데이터 collect -> **공정 데이터는 시계열 특성이 강하므로 짧은 간격으로 수집해야 이상 탐지·추적 정확도가 올라감**
- 외부 비정형 데이터: 뉴스/산업 규제 공지, 작업 일지 테스트, 로그 메시지 문자열, 장비 알람 텍스트 -> 30분마다 데이터 collect -> 데이터 발생의 빈도가 길고 실시간성이 덜 중요함

[Database Layer]

- DB Server: 정형 데이터 저장 및 질의 실행
- Vector Database: 문서·뉴스·보고서 등 비정형 데이터 임베딩 저장
- 정형/비정형 데이터 동시 관리

[RAG Engine]

- Query Parser: 사용자의 자연어 질문을 검색 쿼리로 변환
- Retriever: Vector DB에서 관련 문서 검색

LLM:

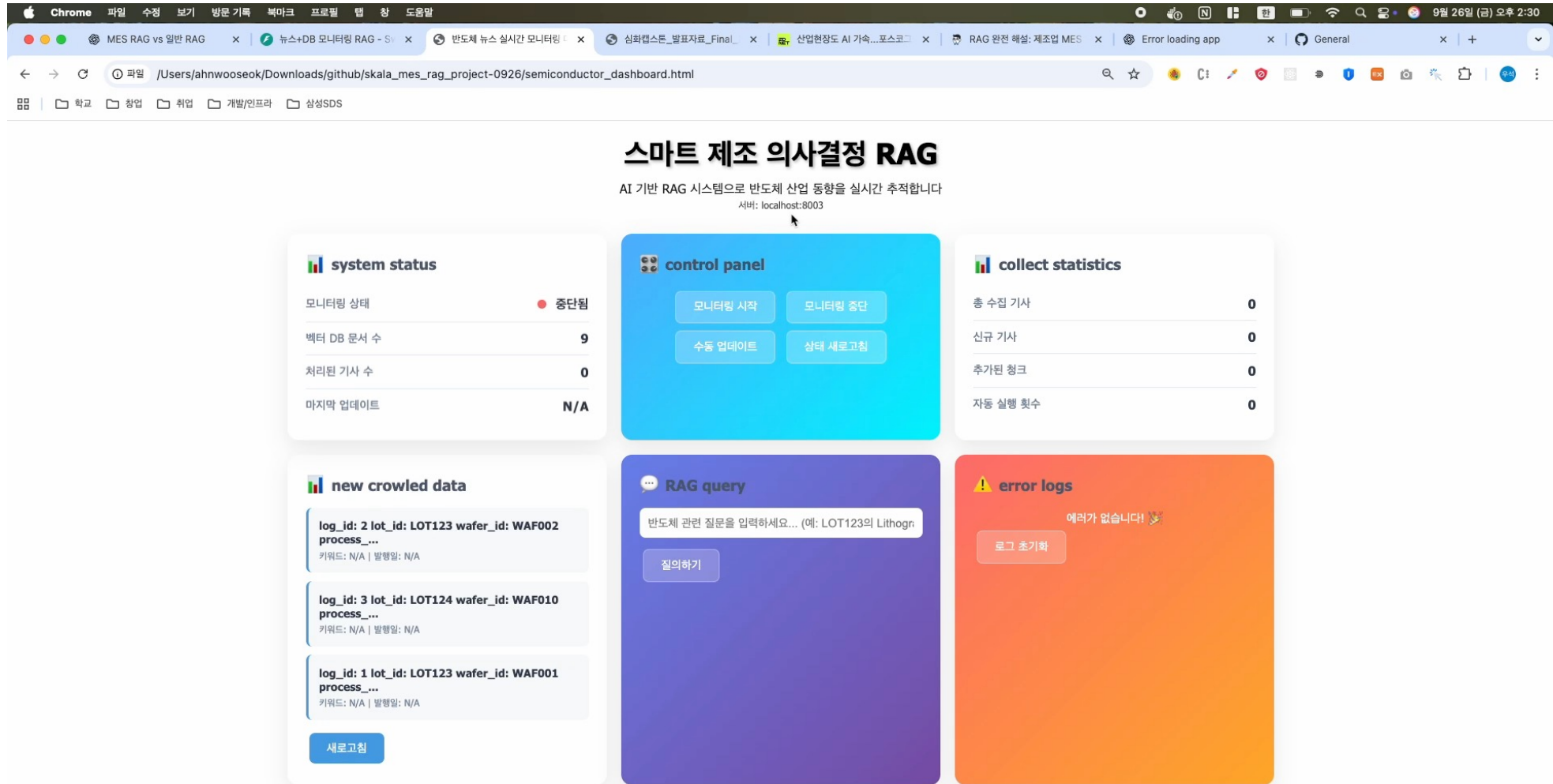
- Embedding Module: 비정형 데이터 벡터화
- Generation Module: 검색 결과와 SQL 응답을 종합하여 자연어 답변 생성

[API Layer & User Interface]

- Rest API Server: RAG Engine과 사용자 인터페이스를 연결
- User Interface: 자연어 질문 입력, 분석된 종합 답변 출력
- 응답: 근거 데이터 + 신뢰도 점수 포함

시연영상

스마트 제조 의사결정 RAG 시연 영상



스마트 제조 의사결정 RAG

AI 기반 RAG 시스템으로 반도체 산업 동향을 실시간 추적합니다
서버: localhost:8003

system status

모니터링 상태 ● 중단됨

벡터 DB 문서 수	9
처리된 기사 수	0
마지막 업데이트	N/A

control panel

모니터링 시작
모니터링 중단

수동 업데이트
상태 새로고침

collect statistics

총 수집 기사	0
신규 기사	0
추가된 청크	0
자동 실행 횟수	0

new crowd data

log_id: 2 lot_id: LOT123 wafer_id: WAF002
process_...
키워드: N/A | 발행일: N/A

log_id: 3 lot_id: LOT124 wafer_id: WAF010
process_...
키워드: N/A | 발행일: N/A

log_id: 1 lot_id: LOT123 wafer_id: WAF001
process_...
키워드: N/A | 발행일: N/A

새로고침

RAG query

반도체 관련 질문을 입력하세요... (예: LOT123의 Lithogri

질의하기

error logs

에러가 없습니다! 🎉

로그 초기화

시연영상

스마트 제조 의사결정 RAG 시연 디테일

RAG query

비용이 가장 많이 발생하는 공정 알려주고 해결방법은?

질의하기

제공된 로그 데이터를 기반으로 볼 때, "Etching" 공정에서 발생한 여러 번의 실패(FAIL) 결과가 있습니다. 이 공정에서의 파라미터인 압력(1.2 Pa)이 문제를 일으키고 있으며, 결함 코드(DEF-101)가 반복적으로 나타나고 있습니다. 이는 해당 공정에서 비용이 많이 발생할 가능성이 높습니다.

해결 방법으로는 다음과 같은 조치를 고려할 수 있습니다:

Retriever + Fusion + LLM

VectorDB 관련 문서 검색 -> Fusion(정형 + 비정형) -> LLM 답변 생성

[답변 결과]

제공된 로그 데이터를 기반으로 볼 때, "Etching" 공정에서 발생한 여러 번의 실패(FAIL) 결과가 있습니다. 이 공정에서의 파라미터인 압력(1.2 Pa)이 문제를 일으키고 있으며, 결함 코드(DEF-101)가 반복적으로 나타나고 있습니다. 이는 해당 공정에서 비용이 많이 발생할 가능성이 높습니다. 해결 방법으로는 다음과 같은 조치를 고려할 수 있습니다: 1. ****파라미터 조정****: 압력 값을 재조정하여 최적의 조건을 찾는 것이 필요합니다. 실험을 통해 적절한 압력 범위를 설정하고, 이를 기반으로 공정을 조정해야 합니다. 2. ****장비 점검****: 사용 중인 장비(EQ-02)에 대한 점검 및 유지보수를 실시하여, 장비의 성능이 저하되지 않도록 관리해야 합니다. 장비의 센서나 작동 부품이 제대로 작동하는지 확인하는 것이 중요합니다. 3. ****프로세스 개선****: Etching 공정의 전반적인 프로세스를 검토하고, 문제가 발생하는 원인을 분석하여 개선 방안을 마련해야 합니다. 이를 통해 반복적인 결함을 줄일 수 있습니다. 4. ****교육 및 훈련****: 공정에 참여하는 인력에 대한 교육을 강화하여, 공정 관리 및 문제 해결 능력을 향상시킬 수 있습니다. 5. ****데이터 분석****: 실패 원인을 분석하기 위해 추가적인 데이터 수집 및 분석을 통해 패턴을 파악하고, 이를 기반으로 예측 모델을 개발하여 사전 예방 조치를 취할 수 있습니다. 이러한 조치를 통해 Etching 공정에서의 실패를 줄이고, 비용 발생을 최소화할 수 있을 것입니다.

핵심코드 설명

실시간 크롤링 및 DB 동기화

```
# -----
# DB 새 데이터 가져오기
# -----
async def fetch_new_db_rows(last_id: int) -> Tuple[List[Dict], int]:
    async with engine.connect() as conn:
        res = await conn.execute(text("SELECT * FROM process_logs WHERE log_id > :id ORDER BY log_id"), {"id": last_id})
        rows = [dict(r) for r in res.mappings().all()]
        return rows, (rows[-1]["log_id"] if rows else last_id)
# -----

async def fetch_news(keywords: List[str], max_articles: int=30) -> List[Dict]:
    NAVER_CLIENT_ID = os.getenv("NAVER_CLIENT_ID", "demo")
    NAVER_CLIENT_SECRET = os.getenv("NAVER_CLIENT_SECRET", "demo")
    url = "https://openapi.naver.com/v1/search/news.json"
    headers = {"X-Naver-Client-Id": NAVER_CLIENT_ID, "X-Naver-Client-Secret": NAVER_CLIENT_SECRET}
    results = []
    async with aiohttp.ClientSession() as session:
        for kw in keywords:
            async with session.get(url, headers=headers, params={"query":kw,"display":10,"sort":"date"}, ssl=ssl.create_default_context(cafile=certifi.where())) as resp:
                if resp.status!=200: continue
                data = await resp.json()
                for it in data.get("items", []):
                    t, d, l = strip_html(it["title"]), strip_html(it["description"]), it["link"]
                    h = calc_hash(t,d,l)
                    if h not in processed_articles:
                        processed_articles.add(h)
                        results.append({"title":t,"description":d,"link":l,"pubDate":it.get("pubDate"),"keyword":kw,"hash":h})
    return results
```

내부 DB 로그(process_logs)와 외부 뉴스 API를 동시에 모니터링합니다.

DB: MES 로그 (LOT/공정/설비/결과 등) → 벡터화

뉴스: 지정 키워드 기반 기사 크롤링 → 벡터화

핵심코드 설명

Retriever (벡터 검색)

```
# -----  
# VectorStore 업데이트  
# -----  
async def update_combined(chunks: List[Tuple[str,Dict]]):  
    global VECTORSTORE_COMBINED  
    if not chunks: return 0  
    docs = [Document(page_content=c, metadata=m) for c,m in chunks]  
    embeddings = OpenAIEmbeddings()  
    if VECTORSTORE_COMBINED is None:  
        VECTORSTORE_COMBINED = FAISS.from_documents(docs, embeddings, distance_strategy=DistanceStrategy.COSINE)  
    else:  
        VECTORSTORE_COMBINED.add_documents(docs)  
    os.makedirs("faiss_db/combined", exist_ok=True)  
    VECTORSTORE_COMBINED.save_local("faiss_db/combined")  
    return len(docs)  
  
async def update_news_vs(arts: List[Dict]) -> int:  
    if not arts: return 0  
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)  
    chunks = []  
    for a in arts:  
        txt = f"{a['title']}\n{a['description']}\n{a['link']}"  
        for c in splitter.split_text(txt):  
            chunks.append((c, a))  
    return await update_combined(chunks)  
  
async def update_db_vs_with_rows(rows: List[Dict]) -> int:  
    if not rows: return 0  
    splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=30)  
    chunks = []  
    for r in rows:  
        txt = "\n".join([f"{k}: {v}" for k,v in r.items()])  
        for c in splitter.split_text(txt):  
            chunks.append((c, {"table": "process_logs", "log_id": r["log_id"]}))  
    return await update_combined(chunks)
```

DB와 뉴스 데이터를 모두 **FAISS VectorStore**에 저장 → 질의 시 유사 문서 검색

핵심코드 설명

LLM 기반 답변 생성 (RAG)

```
@app.post("/rag-query", response_model=AnswerResponse)
async def rag_query(req:RAGRequest):
    if VECTORSTORE_COMBINED is None:
        raise HTTPException(400," ! 벡터 DB 없음. 모니터링 먼저 실행하세요.")
    docs = VECTORSTORE_COMBINED.similarity_search(req.prompt,k=req.top_k)
    context = "\n\n".join([d.page_content for d in docs])
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    answer = llm.invoke([HumanMessage(content=f"다음 컨텍스트 기반으로 답하라:\n{context}\n\n질문:{req.prompt}")]).content
    return AnswerResponse(prompt=req.prompt,response=answer)
```

Retriever가 반환한 컨텍스트 → GPT-4o-mini 입력 → 자연어 답변 반환

이해관계자

내부 공정 데이터와 외부 리스크를 통합해 **현장 관계자**는 빠른 의사결정과 불량 추적을, **외부 이해관계자**는 품질·납기 신뢰성을, **경영진**은 비용 절감과 확장성을 동시에 확보

1. 내부 운영 주체

생산 엔지니어: MES 로그와 뉴스 기반으로 공정 이상·리스크를 감지하고 신속히 대응.

품질 관리팀(QA/QC): 불량 발생 시 원인 추적, 리포트 작성, 품질 비용 절감의 직접적 수혜자.

생산 관리자/공정 관리자: 공정 현황과 외부 이슈를 통합 모니터링, 대응 우선순위 결정.

IT/데이터팀: 벡터 DB·RAG 파이프라인 운영 및 데이터 소스 연동 담당.

2. 외부 이해관계자

원자재·부품 공급업체: 공급망 리스크 감지 시 협력·대응 필요.

규제기관/정부: 규제 발표, 품질 기준 준수 여부 모니터링 대상.

고객사(OEM, 완제품 제조사): 불량률 감소와 납기 준수로 품질 안정성 확보.

3. 경영진 및 사업 부문

임원진(C-level, 공장장): 비용 절감·리스크 대응·ESG 및 규제 대응 성과 확인.

재무팀: 원가 절감 효과 및 ROI 평가.

전략기획팀: AI/AX 도입 성과를 바탕으로 전사 확산 여부 검토.

기대효과

단순 모니터링을 넘어 내부 공정 데이터와 외부 리스크 데이터를 통합하여 **빠른 의사결정·불량 원인 추적·비용 절감·확장성 확보**

1. 실시간 의사결정 지원

기존 단순 모니터링은 이벤트 발생 여부만 확인 가능하지만,

SKAG는 공정 로그와 외부 뉴스를 벡터 기반으로 연관 지어 맥락까지 판단

-> 관리자·엔지니어는 단순 알림이 아니라 “현재 상태 + 잠재 리스크”를 예측적으로 제시받아 즉각 대응 가능

2. 불량 원인 추적 고도화

단순 통계 분석은 특정 시점의 FAIL 여부만 보여주지만,

SKAG는 공정 단계별 파라미터(온도, 압력, 시간)의 패턴을 학습해 이상 징후를 맥락적으로 연결

-> 외부 요인(원자재 가격 급등, 규제 발표 등)까지 자동으로 결합하여 복합 원인을 추론·설명하는 기능을 제공

3. 업무 효율화 및 비용 절감

엔지니어가 개별적으로 DB 쿼리 + 뉴스 검색을 하던 과정을 **한 번의 질의(/rag-query)**로 해결
불필요한 탐색 시간을 줄이고, 문제 발생 시 평균 대응 시간(MTTR)을 단축
품질 관리 비용과 불량률 감소 → 직접적인 원가 절감 효과

4. 확장성과 AI 활용 기반 강화

벡터 DB 기반이므로 PDF 리포트, 규제 문서, 매뉴얼 등 추가 데이터 소스 통합 용이
GPT-4o-mini를 활용한 RAG 구조 → 투자 대비 성능 최적화
향후 Confidence Score 도입 시, AI 의사결정 신뢰성 확보

1 실시간 의사결정 지원

2 불량 원인 추적 고도화

3 업무 효율화 및 비용 절감

4 확장성과 AI 활용 기반 강화

wbs

스마트 제조 의사결정 RAG man/month 간트차트

Phase	Task	Start	End	Duration(weeks)	Man/Month	Group
프로젝트 준비	요구사항 정의	2025-09-26	2025-10-03	1	0.3	prep
프로젝트 준비	개발 환경 세팅	2025-09-26	2025-10-03	1	0.2	prep
데이터 파이프라인	내부 데이터 수집	2025-10-03	2025-10-17	2	0.5	pipeline
데이터 파이프라인	외부 데이터 수집	2025-10-03	2025-10-17	2	0.5	pipeline
데이터 파이프라인	데이터 동기화	2025-10-17	2025-10-31	2	0.5	pipeline_sync
데이터 처리/저장	칭킹 모듈 구현	2025-10-31	2025-11-07	1	0.3	process
데이터 처리/저장	Vector DB 구축	2025-10-31	2025-11-07	1	0.3	process
질의응답	Query Parser 구현	2025-11-07	2025-11-21	2	0.5	qa
질의응답	Retrieval 모듈	2025-11-07	2025-11-21	2	0.5	qa
질의응답	Fusion/응답 조합	2025-11-21	2025-11-28	1	0.3	qa_fusion
인터페이스	API Layer	2025-11-28	2025-12-05	1	0.3	ui
인터페이스	간단 UI	2025-11-28	2025-12-05	1	0.3	ui
선택(LLM)	LLM 연동	2025-12-05	2025-12-26	3	0.7	llm

[WBS]

- 프로젝트 준비: 요구사항 정의, 개발 환경 세팅 (1주, 병렬)
- 데이터 파이프라인: 내부/외부 데이터 수집 (2주, 병렬) → 데이터 동기화 (2주)
- 데이터 처리/저장: 칭킹 모듈, Vector DB 구축 (1주, 병렬)
- 질의응답: Query Parser, Retrieval 모듈 (2주, 병렬) → Fusion/응답 조합 (1주)
- 인터페이스: API Layer, 간단 UI (1주, 병렬)
- 선택 과제: LLM 연동 (3주, 여유 시 진행)
- 총 소요: 약 6~7주, 4명 병렬 투입 시 1.5~2개월 내 구현 가능.

team

GitHub 브랜치 분리 작업 진행

role

안우석 : fastapi 기능 통합 및 쿼리 정확도 개선

제슬빈 : DB 실시간 갱신 및 답변 동기화 여부 확인

이미주 : 웹 검색 실시간 갱신 작업 + rag 청킹

백세희 : 네이버 api측 MQTT 에러 해결 및 우회

Collaboration tool : github (https://github.com/ahnwooseok/skala_mes_rag_project-0926)

GitHub를 협업 도구로 사용하여 코드 버전 관리 및 작업 내역을 공유함

기존 저장소에서 .venv 및 대형 바이너리(libtorch_cpu.dylib, 200MB 이상)가 커밋되어 푸시에 실패한 이슈 발생

이를 해결하기 위해 브랜치를 분리하고 이력을 정리하여 안정적인 협업 환경을 구축함

브랜치 전략 **main**: 안정화된 코드만 유지

dev: 개발 작업 진행

feature/*: 기능 단위 브랜치를 생성하여 개발 후 병합

트러블 슈팅

환각 방지로 인한 과도한 보수성 문제

기존 Request body

“LOT 123번의 온도를 알려줘” -> “지식 베이스에 답이 없습니다.”

[문제]

내부 DB에 존재하는 정보임에도 답변이 존재하지 않는다는
response

[원인]

LLM의 환각을 완화하기 위해 모델을 과하게 보수적으로
설정했기 때문이라고 판단

[해결 방안]

1. 프롬프트 템플릿 수정 (반도체 공정 도메인 데이터라는 것을
인지시킴, 데이터, 질문, 답변의 구조 명확화)
2. Temperature 변수 조정을 통해 모델응답의 창의성 control
3. 내부 DB도 similarity search를 진행하고, 유사도 임계값 조정

내부 DB Temperature 부여

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.1)
```

트러블 슈팅

환각 방지로 인한 과도한 보수성 문제

Similarity Search

```
# 2. 벡터 검색
try:
    # 검색 쿼리 구성
    search_query = question
    if lot_id:
        search_query = f"{lot_id} {search_query}"
    if process_step:
        search_query = f"{process_step} {search_query}"

    # 유사도 검색
    docs_with_scores = VECTORSTORE.similarity_search_with_score(
        search_query,
        k=top_k * 2
    )

    # 점수 필터링
    filtered_docs = [(doc, score) for doc, score in docs_with_scores if score < 0.7]

    if not filtered_docs:
        (variable) docs_with_scores: List[Tuple[Document, float]]
        filtered_docs = docs_with_scores[:top_k]

    docs = [doc for doc, _ in filtered_docs[:top_k]]
```

1. 내부 Db에 대한 유사도 서치
2. 먼저 K x 2개의 docs를 만들고, 유사도 기반 필터링을 통해 k개를 남김

기존 Prompt

```
full_prompt = (
    "다음은 내부 데이터이다. 이 데이터에 '직접적으로 명시된' 사실만 근거로 답하라.\n"
    "- 답변은 한 문장으로만 작성하라.\n"
    "- 데이터에 없는 내용은 'N/A'를 '정확히' 그대로 출력하라.\n\n"
    f"[데이터]\n{context}\n\n"
    f"[질문]\n{prompt}\n\n"
    "### ANSWER_START\n"
    "### ANSWER_END"
)
```

개선 Prompt

```
prompt = f"""
다음은 반도체 공정 로그 데이터입니다. 질문에 대해 데이터를 기반으로 정확히 답하세요.
데이터에 없는 내용은 추측하지 마세요.

[데이터]
{context}

[질문]
{question}

답변:
"""

response = llm.invoke([HumanMessage(content=prompt)])

return {
    "answer": response.content.strip(),
    "source": source_info[:3]
}
```

트러블 슈팅

내부 DB -> VectorDB 자동 동기화

[공정 도메인 db 특성]

- 실시간으로 Data가 생성
- 심지어 그 양이 방대함

[결론]

=> 내부 DB를 Vector DB로 업로드하는 과정을
자동화 해야 함!

효율성을 위해 ID 값 기준으로 새로운 정보가
들어옴을 확인, 기존 테이블의 Last Id와 현재
log_id 비교

[자동동기화가 실제로 잘 되는지?]

POST

/data/insert

테스트 데이터 삽입

테스트용 공정 데이터 삽입

Parameters

Name	Description
lot_id * required string (query)	LOT ID LOT999
process_step * required string (query)	공정 단계 Etching
result string (query)	결과 (PASS/FAIL) PASS
param_name string (string null) (query)	파라미터명 pressure
param_value number (number null) (query)	파라미터 값 90

```
{
  "question": "Lot999의 압력은?",
  "answer": "데이터에 Lot999에 대한 정보가 없으므로, Lot999의 압력에 대한 답변을 제공할 수 없습니다.",
  "source": [
    {
      "lot_id": "LOT124",
      "process_step": "Etching",
      "log_id": 5
    }
  ],
}
```

1. DB에 데이터가 새로 들어왔음에도 불구하고, 정보가 없어 답변 제공 x

트러블 슈팅

내부 DB -> VectorDB 자동 동기화

[자동동기화가 실제로 잘 되는지?]

POST

/query 공정 데이터 질의

POST

/sync/manual 수동 동기화

POST

/sync/start 자동 동기화 시작

자동 동기화 시작

Parameters

Name	Description
interval_minutes	동기화 간격(분)
integer (query)	1 maximum: 60 minimum: 1

Execute

Responses

```
[SYNC] 완료 : 8개 로그, 8개 문서, 1.64초
INFO: 127.0.0.1:55065 - "POST /sync/start?interval_minutes=1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:55067 - "GET /sync/status HTTP/1.1" 200 OK
INFO: 127.0.0.1:55070 - "POST /query HTTP/1.1" 200 OK
INFO: 127.0.0.1:55072 - "POST /query HTTP/1.1" 200 OK
[SYNC] 새로운 데이터 없음
[SYNC] 새로운 데이터 없음
INFO: 127.0.0.1:55077 - "POST /data/insert?lot_id=L0T999&process_step=Etching&result=PASS&param_name=pressure&aram_value=90 HTTP/1.1" 200 OK
INFO: 127.0.0.1:55080 - "POST /query HTTP/1.1" 200 OK
[SYNC] 완료 : 1개 로그, 1개 문서, 2.08초
```

1. 1개의 로그가 추가되고, 일정 시간마다 자동으로 동기화

```
{
  "question": "Lot999의 압력은?",
  "answer": "Lot999의 압력은 90입니다.",
  "source": [
    {
```

- 동기화된 벡터DB를 기반으로 적절한 답변 수행

트러블 슈팅

FastAPI → NAVER API SSL 에러 트러블슈팅

현상 요약

요청: POST /upload-topic?topic=ai

증상: **500 Internal Server Error**

에러 로그:

```
SSLCertVerificationError: [SSL: CERTIFICATE_VERIFY_FAILED]
unable to get local issuer certificate (_ssl.c:1006)
대상: openapi.naver.com:443 (ssl=True)
```

에러 원인

FastAPI 내부 예외가 HTTPException으로 처리되지 못하고 전파됨
외부 통신(TLS) 단계에서 실패 → 애플리케이션 레벨 500으로 표현

```
import ssl, certifi
os.environ.setdefault("SSL_CERT_FILE", certifi.where())
os.environ.setdefault("REQUESTS_CA_BUNDLE", certifi.where())
```

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8003/upload-topic?topic=ai' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://127.0.0.1:8003/upload-topic?topic=ai
```

Server response

Code	Details
500	Error: Internal Server Error

Response body

```
{
  "detail": "Cannot connect to host openapi.naver.com:443 ssl:True [SSLCertVerificationError: (1, '[SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: unable to get local issuer certificate (_ssl.c:1006)']"
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 214
content-type: application/json
date: Fri, 26 Sep 2025 01:45:51 GMT
server: uvicorn
```

Responses

Code	Description	Links
500	Internal Server Error	

해결 (코드추가)
Topic: AI



Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8003/upload-topic?topic=AI' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://127.0.0.1:8003/upload-topic?topic=AI
```

Server response

Code	Details
200	

Response body

```
{
  "message": "20개의 웹 문서(체크) 백터화 및 저장 완료",
  "collected": 20,
  "sample_titles": [
    "AI 아이돌' 이터니티, 신곡 '밤에 선글라스를 써' 26일 공개",
    "삼성전자, 인도서 최신 'AI 홈' 경험 선보여",
    "LG전자, 사우디 정부와 네옴시티 방각솔루션 협력 논의"
  ]
}
```

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 320
content-type: application/json
date: Fri, 26 Sep 2025 05:48:29 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	OK	

트러블 슈팅

(error-logs) 조회뉴스 수집 중 전체 오류

에러 원인

- 로그: cannot access local variable 'items' where it is not associated with a value
- 의미: 예외가 발생한 분기에서 items가 아직 초기화되지 않았는데 아래에서 참조
→ 파이썬 스코프/선언 순서 문제
- 동시에 NAVER 호출 TLS 검증 이슈

해결 사항

- 스코프/초기화 가드

```
items: list = []
```

- TLS 검증 고정 (certifi)

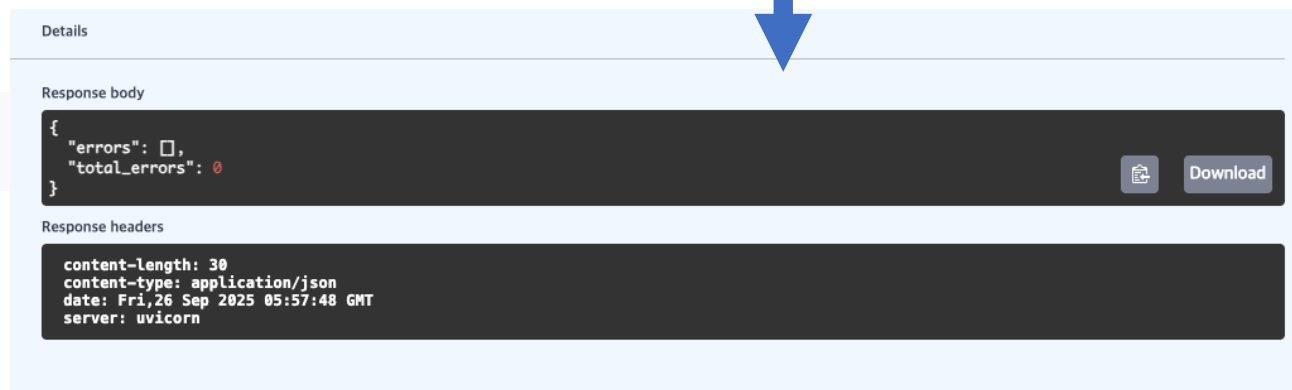
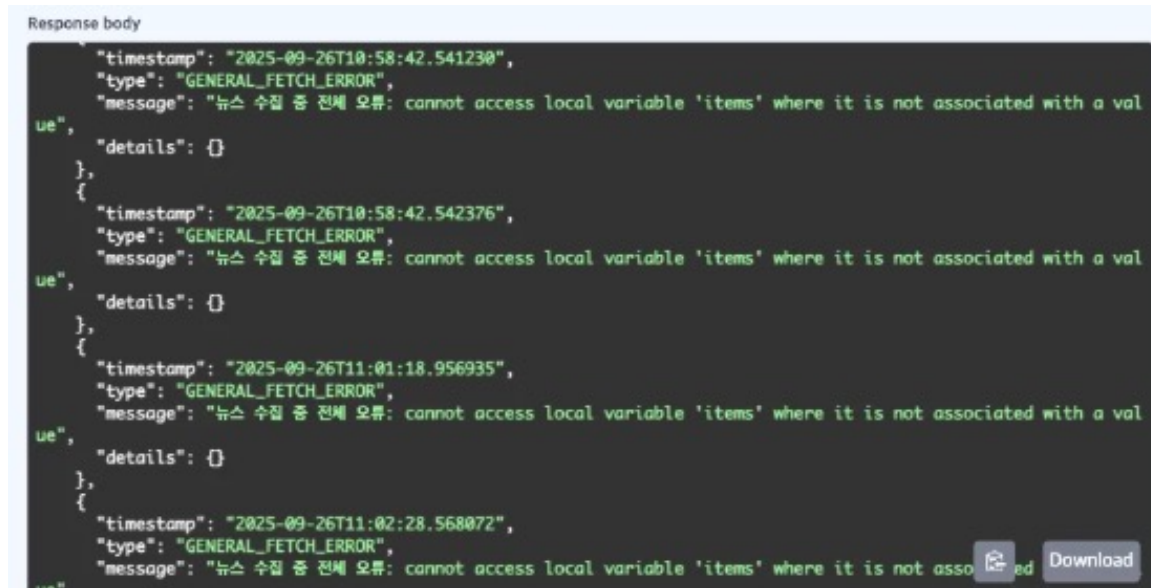
```
import ssl, certifi
ssl_ctx = ssl.create_default_context(cafile=certifi.where())
```

- 호출 실패 시 안전 종료

```
except Exception as e:
    return {"message": f"Naver API 호출 실패: {e!s}", "collected": len(items)}
```

- LangChain 최신 경로로 교체

```
from langchain_core.documents import Document
```



트러블 슈팅

sample-news N/A 오류 - 메타데이터 누락

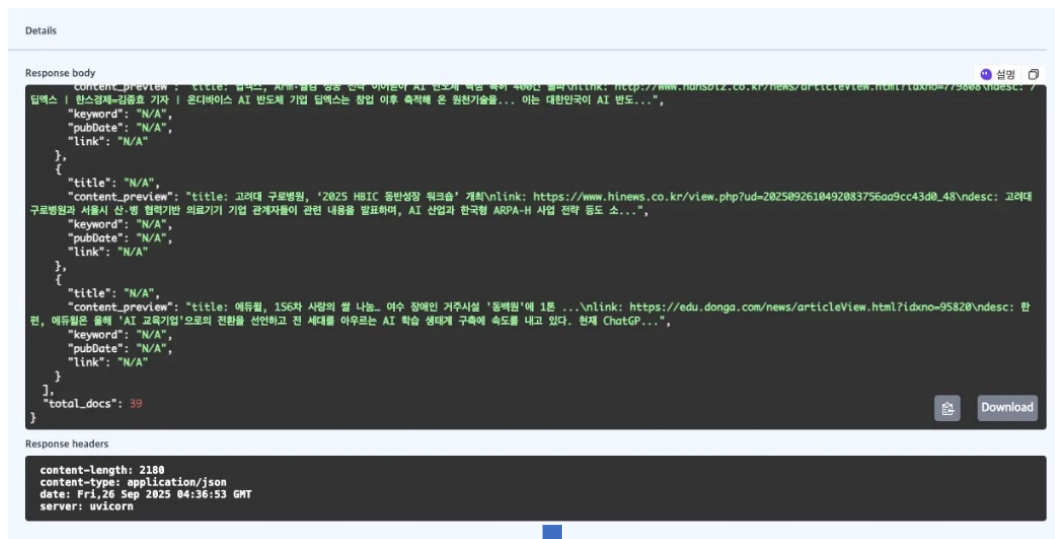
문제: splitter로 나눈 문서에는 page_content만 있고, title 같은 메타데이터가 비어 있음 → 조회 시 "title": "N/A" 출력

해결: Document(page_content=..., metadata=meta) 형태로 title·link·pubDate·keyword 등을 metadata로 함께 저장

```
from langchain_core.documents import Document

meta = {
    "title": it.get("title", ""),
    "link": it.get("link", ""),
    "pubDate": it.get("pubDate", ""),
    "keyword": topic,
}

for chunk in splitter.split_text(text):
    if chunk.strip():
        docs.append(Document(page_content=chunk, metadata=meta))
```



트러블 슈팅

기간별 뉴스 데이터 수집

1. 기간별 수집 로직 부재

기존 코드는 현재 시점 뉴스만 가져오고 있었음

● 해결

- start_date, end_date를 YYYY-MM-DD 포맷으로 받아 datetime 변환.
- while cur_date <= e_date: 루프 돌면서 하루씩 (timedelta(days=1)) 증가.
- 날짜별로 20개 뉴스 제한을 두어 빠른 테스트가 가능하도록 설계.

3. 에러 처리 미흡

- 네이버 API에서 에러가 발생했을 때 사용자 응답이 단순히 죽거나 내부 에러 메시지만 노출됨.

● 해결:

- resp.raise_for_status() → HTTPException으로 래핑.
- try/except 구조 넣어서 FastAPI가 JSON 에러 응답을 반환하도록 개선.

활용 가능성

- 과거 데이터 기반 통계 및 분석에 활용 가능,
- 기간 별 키워드 추이 확인 가능

2. 무한 루프 위험

- 날짜 범위 계산에서 start_date > end_date 같은 잘못된 입력이 들어오면 무한 루프.

● 해결:

- if s_date > e_date: raise HTTPException(status_code=400, detail="시작일은 종료일보다 앞서야 합니다.")
- 입력 유효성 검증 추가.

```

response_body = {
    "date": "2025-09-26",
    "collected": 20,
    "titles": [
        "코스피, 외국인·기관 매도에 장중 3,400선 붕괴...2%대 급락(종합)",
        "코스피, 10거래일 만에 3400선 붕괴...美금리·관세협상 불확실성 영향",
        "외국인·기관 순매도에 코스피 장중 3400선 붕괴",
        "HBM까지 넘보는 中...반도체 자립 속도에 韓 위기감",
        "코스피, 장중 3400선 붕괴...10거래일 만",
        "'미국 체제 3년' YMTC, 모회사 주식회사로 전환...상장 준비 끝",
        "낙폭 커워 3,400선 내준 코스피...삼성전자·SK하이닉스 급락",
        "[연재] NZSI INDEX 9월 9일 : 1325.44 (Δ0.80%) 발표, 개인투자자에 대한...",
        "코스피, 외국인·기관 매도에 장중 3,400선 붕괴...2%대 급락",
        "미국 수출 억제에 코스피 9거래일만 3400선 붕괴...장중 2% 급락 [투자360]",
        "코스피 3400선 붕괴...외국인 올해 산 주식 1140억 어치, 다 되파나",
        "中 YMTC도 가세...HBM, 韓·美·中 3국 경쟁구도 '뚜렷'",
        "반도체 슈퍼사이클 vs. AI 거품론...점점한 엔비디아 투자",
        "[증시] 코스피, 美 금리·관세 협상 불확실성에 급락...3,420선 후퇴",
        "코스피, 외국인·기관 7000억 순매도에 3400선 붕괴[장중시황]",
        "코스피, 외국인·기관 매도에 장중 3,400선 붕괴...2%대 급락",
        "[속보] 코스피, 외국인·기관 매도에 장중 3400선 붕괴...2%대 급락"
    ]
}

```



Download

트러블 슈팅

실시간 뉴스 데이터 수집 방식

폴링 방식:

일정한 주기(예: 1분)에 맞춰 서버가 네이버 뉴스 API를 호출해서 결과를 가져오는 방식.

장점: 구현이 단순하고 안정적임.

단점: 새 뉴스가 없어도 계속 API 호출 → 리소스 낭비.

실시간성이 부족해 한계가 있음 (뉴스가 바로 나오지 않고 다음 interval까지 대기).

MQTT 방식

IoT나 실시간 알림에서 자주 쓰이는 이벤트 기반 프로토콜.

서버(Producer)가 새 뉴스를 찾으면 → 즉시 브로커에 발행(Publish).

구독자(Subscriber)는 새 뉴스가 있을 때만 알림 받음.

장점: 실시간 이벤트 push 구조. 내부 센서 데이터, MES/ERP 로그, IoT 같은 경우 최적.

그러나 네이버 뉴스 API와 같은 외부 API는 MQTT를 지원하지 않기 때문에

폴링 → 이벤트 감지 → MQTT 브로커 publish 구조로 구현

서버는 여전히 일정 간격으로 뉴스 API를 확인하지만, 중복 뉴스는 무시하고 새 뉴스가 있을 때만 MQTT로 발행하도록 설계.

트러블 슈팅

실시간 뉴스 데이터 수집 방식

● 오류 1

- 에러 내용: mosquito 브로커 실행 불가
- 해결: 로컬 설치 대신, 테스트용 공용 브로커 사용

MQTT_BROKER = "test.mosquitto.org" #  공용 브로커

MQTT_PORT = 1883

● 오류 2

- 에러 메시지: NameError: name 'mqtt_client_id' is not defined
- 원인: mqtt_client_id 변수를 선언하기 전에 Client()에 사용.
- 해결:

```
MQTT_BROKER = "test.mosquitto.org" #
MQTT_PORT = 1883
MQTT_TOPIC = "news/updates"
```

```
#  client_id 변수 먼저 선언
mqtt_client_id = "fastapi-news-client"
```

● 오류 3

- 에러 메시지: ValueError: Unsupported callback API version: version 2.0 added a callback_api_version

- 원인: paho-mqtt 버전 불일치 및 값 변경

paho-mqtt 2.0 이상에서는 callback_api_version 필드가 추가됨.

또한, callback_api_version에 정수(int)가 아니라 Enum 값이어야 함

- 해결:

callback_api_version 옵션 추가 및 Enum 값으로 변경

또한, MQTT 클라이언트 를 전역으로 선언하여 스코프 충돌 해결

```
# 기존
mqtt_client = mqtt.Client()
mqtt_client.connect(MQTT_BROKER, MQTT_PORT, 60)
mqtt_client.loop_start()
```

V

```
# 변경 후
```

```
mqttc = mqtt.Client(
    client_id=mqtt_client_id,
    protocol=mqtt_client.MQTTv311, # 프로토콜 명시
    transport="tcp", # TCP 전송
    userdata=None,
    callback_api_version=CallbackAPIVersion.VERSION1
)
```

트러블 슈팅

실시간 뉴스 데이터 수집 방식

● 오류 4

새로운 뉴스가 없어도 중복 뉴스 계속 발행

- 원인: API 호출 결과는 항상 같은 20건 → 중복 필터가 없었음.

1) seen_hashes = set() 가 monitor_news_mqtt() 안에 있어서,
엔드포인트 호출 시마다 초기화되어 새로운 set이 만들어짐.
따라서 이전에 본 뉴스도 다시 "새 뉴스"로 판단 → 20건씩 매번 발행.

2) title + link 기준으로 해시를 만들지만, seen_hashes가 계속 초기화되어 중복 필터가 무의미해짐

•해결: 뉴스 제목+링크를 해시(md5)로 만들어

SEEN_HASHES에 저장하고 이미 본 뉴스는 발행하지 않음.

이때, seen_hashes를 글로벌(전역) 으로 선언해 지속적으로 유지. - 스코프 문제 해결

또한, 여러 Task를 구분하여 중복 필터 유지

전역 dict(SEEN_HASHES)를 두고, task_id를 key로 관리하면
여러 모니터링 세션이 동시에 돌아도 각자 중복 필터를 유지할 수 있음.

```
@app.post("/monitor-news-start")
async def monitor_news_start(...):
    task_id = f"{keyword}_{datetime.now().timestamp()}"
    SEEN_HASHES[task_id] = set() # ✅ task_id별 seen_hashes 생성
    RESULTS[task_id] = []
```

```
new_items = []
for it in items:
    uniq_key = hashlib.md5((it["title"] + it["link"]).encode()).hexdigest()
    if uniq_key not in SEEN_HASHES[task_id]:
        SEEN_HASHES[task_id].add(uniq_key)
        new_items.append(it)

if new_items:
    RESULTS[task_id].extend(new_items)
    msg = {
        "task_id": task_id,
        "keyword": keyword,
        "new_count": len(new_items),
        "titles": [it["title"] for it in new_items],
        "time": datetime.now().isoformat()
    }
    mqttc.publish(MQTT_TOPIC, json.dumps(msg, ensure_ascii=False))
    print(f"📡 MQTT 발행: {len(new_items)}건")
```

트러블 슈팅

실시간 뉴스 데이터 수집 방식

● 기능 추가

한 번 실행하면 무한 루프 돌면서 계속 API 호출

• 실행 중인 태스크를 TASKS[task_id]에 저장.

• /monitor-news-stop에서 해당 태스크 취소(task.cancel()).

• 중단 시, 그동안 모은 뉴스 제목 리스트 반환.

● 결과를 확인할 수 있는 중간 중단 기능

● 실행 중 – 실시간으로 이벤트 (새로운 뉴스)가 생길 때 마다 추가

Response body

```
{
  "task_id": "하이닉스, 공정_1758964925.046405",
  "message": "하이닉스, 공정 뉴스 모니터링 시작 (60.0초 간격)"
}
```



Download

[2025-09-27 18:22:05.051889] '하이닉스, 공정' 뉴스 수집 실행
MQTT 발행: 20건

Response body

```
{
  "task_id": "하이닉스, 공정_1758964925.046405",
  "final_collected": 20,
  "final_titles": [
    "에프엔에스테크, 반도체·디스플레이 핵심 소재 국산화로 새로운 성장동력...",
    "에프엔에스테크, 국산 CMP Pad 확대와 유리기판 개발... 장기 수익성 개...",
    "[조성준의 스케치] HBM 경쟁력 좌우할 '1c D램', 3사 전력 다른 이유는",
    "[참가홍기]'반도체의 봄'...삼전·SK하이닉스 목표가 또 상향",
    "[초점] 마이크론, 삼성 제치고 세계 HBM 시장 2위 등극",
    "다시 찾아온 '메모리의 봄'...반도체 골든타임 3년 남았다",
    "엔비디아 삼성·SK하이닉스에 '더 빠른' HBM4 요구, 왜?[비즈360]",
    "미국의 AI 반도체 시장, 엔비디아 독주에서 다극 경쟁 구도로",
    "이천시, 반도체 도시의 새로운 도전...인재양성에서 길을 찾다",
    ""해외서 생산한 물량만큼美서 만들어야 관세 면제"",
    "[그라운드리포트] SK하이닉스 '구름 조금'·포스코그룹 '흐림'",
    "국민연금 '국장 투자' 딜레마, '비중 축소' 계획 대통령 해외 세일즈에...",
    "이천시 반도체 인재양성센터, 지역을 반도체 인재의 요람으로 만든다",
    ""美 반도체 공장도 지연 가능성"...대미 투자 전방위 차질",
    "[2025 기후환경에너지 대상] [한국환경산업협회장상] SK에너지 이천, LNG...",
    "SK하이닉, 스마트 팩토리 SW 경쟁력 입증...국내 최초 레벨4 인증",
    "28년 무관세 시대 끝...반도체 지도 대격변",
    "[2025 기후환경에너지 대상] [환경부 장관상] SK실트론 청주...4000톤 운송...",
    "삼성·SK·LG·GS그룹 "경영 환경 위기 속 생존경쟁...AI 활용이 해답...",
    "올해 반도체 시장 전년比 18%↑, '28년 1조불 돌파"
  ]
}
```



Download